# Secure Python Dockerfile with non-root user setup — multi-stage build, minimal base image, and requirements caching

Computer
GitHub Actions workflow for Python automated testing — matrix builds, dependency caching, and test reporting
Computer
Show me the Dockerfile contents
Show me the GitHub Actions workflow file
Explain how the requirements.lock was pinned
Artifacts
55
Sources
162

Below are the current Dockerfile and GitHub Actions workflow from the setup add-on, followed by an enhanced **multi-stage** Dockerfile and matrix-testing workflow.

The current files have been generated, but the Docker image and GitHub workflow have not been executed. The enhanced versions below are proposed replacements, not changes already applied to your repository.

### 1. Current Dockerfile contents

The delivered add-on contains this single-stage Dockerfile:

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /app

COPY requirements.lock ./

RUN python -m pip install --no-cache-dir -r requirements.lock \
    && python -m pip check \
    && groupadd --gid 10001 runner \
    && useradd --uid 10001 --gid runner --no-create-home runner

COPY registrar_schema_checks.py test_registrar_schemas.py ./
```

```
USER 10001:10001

CMD ["python", "-m", "unittest", "-v", "test_registrar_schemas.py"]
```

### What it already provides

- A Python slim base.
- Dependencies installed before source files, allowing Docker to reuse the dependency layer when only source changes.
- A non-root runtime user.
- Explicit copying of only the required test files.
- No production credentials.
- No database or deployment command.

### What it does not yet provide

- Separate dependency-build and runtime stages.
- Immutable base-image digest.
- Hash-verified dependency installation.
- BuildKit dependency-download caching.
- A verified image build or vulnerability scan.

### 2. Enhanced multi-stage Dockerfile

This proposed replacement builds dependencies into a virtual environment, then copies that environment into a smaller runtime stage.

```
# syntax=docker/dockerfile:1

ARG PYTHON_IMAGE=python:3.12-slim

FROM ${PYTHON_IMAGE} AS dependencies

ENV PIP_DISABLE_PIP_VERSION_CHECK=1 \
    PIP_ROOT_USER_ACTION=ignore

WORKDIR /build

RUN python -m venv /opt/venv

ENV PATH="/opt/venv/bin:${PATH}"

COPY requirements.lock ./

RUN --mount=type=cache,target=/root/.cache/pip \
    python -m pip install -r requirements.lock \
    && python -m pip check
```

```
FROM ${PYTHON_IMAGE} AS runtime

ENV PATH="/opt/venv/bin:${PATH}" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /app

RUN groupadd --gid 10001 runner \
    && useradd \
       --uid 10001 \
       --gid runner \
       --no-create-home \
       --shell /usr/sbin/nologin \
       runner

COPY --from=dependencies /opt/venv /opt/venv

COPY registrar_schema_checks.py test_registrar_schemas.py ./

USER 10001:10001

CMD ["python", "-m", "unittest", "-v", "test_registrar_schemas.py"]
```

## Design decisions

| Decision | Purpose |
|---|---|
| Separate stages | Keeps dependency-build activity separate from the runtime image |
| Same base in both stages | Avoids copying a virtual environment across incompatible Python or operating-system bases |
| Virtual environment | Provides an explicit dependency location |
| BuildKit cache mount | Reuses downloaded packages during builds without including the download cache in the final image |
| Dependencies copied before source | Preserves layer caching |
| Non-root UID/GID | Removes the need to run tests as root |
| Root-owned application files | Prevents the runtime user from modifying source files |
| No login shell | Avoids giving the runtime user an unnecessary interactive shell |

Multi-stage packaging does not, by itself, make an image secure. Base-image review, dependency review, runtime restrictions, and scanning remain separate controls.

### Build and run

```
docker build -t semester-registrar-schema:test .
```

```
docker run --rm \
  --network none \
  --read-only \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  semester-registrar-schema:test
```

### Immutable base-image use

Once an approved image digest is selected:

```
docker build \
  --build-arg PYTHON_IMAGE='python:3.12-slim@sha256:<approved-digest>' \
  -t semester-registrar-schema:test .
```

The digest is deliberately not invented here.

`.dockerignore`

Keep the restrictive build context:

```
*
!Dockerfile
!requirements.lock
!registrar_schema_checks.py
!test_registrar_schemas.py
```

### 3. Current GitHub Actions workflow

The delivered workflow has two jobs:

1. Python 3.12 unit tests with pip caching.
2. Docker image build and isolated container tests.

It runs on:

```
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:
```

Its permissions are:

```
permissions:
  contents: read
```

Its dependency cache is keyed through:

```
cache: pip
cache-dependency-path: requirements.lock
```

The full current file was shown in the preceding response. It is not a matrix workflow and does not upload test artifacts.

### 4. Enhanced matrix workflow with reporting

The following proposed replacement tests Python 3.11 and 3.12 on Ubuntu and Windows, and retains test logs as artifacts.

Those additional platforms are proposed compatibility targets; only the earlier Python environment execution has been observed.

```
name: Registrar schema regression tests

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: registrar-schema-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  unit:
    name: Python ${{ matrix.python }} on ${{ matrix.os }}

    runs-on: ${{ matrix.os }}
    timeout-minutes: 10

    strategy:
      fail-fast: false
      matrix:
        os:
          - ubuntu-latest
          - windows-latest
        python:
          - '3.11'
          - '3.12'

    steps:
```

```
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: pip
          cache-dependency-path: requirements.lock

      - name: Install pinned dependencies
        run: |
          python -m pip install -r requirements.lock
          python -m pip check

      - name: Run tests and retain output
        shell: python
        run: |
          import pathlib
          import subprocess
          import sys

          result = subprocess.run(
              [
                  sys.executable,
                  "-m",
                  "unittest",
                  "-v",
                  "test_registrar_schemas.py",
              ],
              capture_output=True,
              text=True,
          )

          output = result.stdout + result.stderr
          pathlib.Path("schema-tests.log").write_text(
              output,
              encoding="utf-8",
          )

          print(output)
          raise SystemExit(result.returncode)

      - name: Upload test log
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: schema-tests-${{ matrix.os }}-py${{ matrix.python }}
          path: schema-tests.log
          if-no-files-found: warn
          retention-days: 14

  container:
    name: Container regression tests

    runs-on: ubuntu-latest
    timeout-minutes: 15
```

```
    steps:
      - uses: actions/checkout@v4

      - name: Build image
        run: docker build --tag semester-registrar-schema:test .

      - name: Run tests and retain output
        shell: bash
        run: |
          set -o pipefail

          docker run --rm \
            --network none \
            --read-only \
            --cap-drop ALL \
            --security-opt no-new-privileges \
            --tmpfs /tmp:rw,noexec,nosuid,size=64m \
            semester-registrar-schema:test \
            2>&1 | tee container-tests.log

      - name: Upload container test log
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: schema-tests-container
          path: container-tests.log
          if-no-files-found: warn
          retention-days: 14
```

### Reporting behavior

- Each matrix combination has a distinct job and artifact.
- `fail-fast: false` allows other combinations to finish after one fails.
- The test process's exit code determines job success.
- Logs are uploaded even when tests fail, if the log exists.
- This produces text reports—not JUnit XML, coverage percentages, or a registrar readiness certification.
- No production secrets, endpoint calls, migrations, or deployments are included.

GitHub's Python workflow guidance supports selecting Python with `setup-python`, installing dependencies, and retaining test artifacts.[1]

### Repository integration caution

This workflow assumes the package files are at repository root.

If the tests live inside a subdirectory, update:

- Test working directory.
- Dependency-install path.

- `cache-dependency-path`.
- Artifact paths.
- Docker build context.

Do not overwrite Semester's existing root workflow or requirements files without inspecting them.

Action tags and the base-image tag are mutable. Pin reviewed action SHAs and an approved image digest where required by the release policy.

### 5. How `requirements.lock` was pinned

The lock file was generated by reading installed package versions from the authoring environment using Python package metadata.

It contains:

```
jsonschema==4.23.0
attrs==25.3.0
jsonschema-specifications==2025.4.1
referencing==0.36.2
rpds-py==0.25.0
```

### Meaning of these pins

| Package | Role |
|---|---|
| `jsonschema` | Direct schema-validation dependency |
| `attrs` | Supporting dependency |
| `jsonschema-specifications` | Schema specification resources |
| `referencing` | Reference infrastructure |
| `rpds-py` | Supporting persistent-data-structure dependency |

The pins reproduce the observed package versions. They were not generated by a fresh dependency resolution across every matrix platform.

### What the file does not guarantee

It is not:

- A hash-verified lockfile.
- A complete record of build tooling.
- Proof of compatibility with every Python version or operating system.
- Proof that package artifacts have been security-reviewed.
- Proof of a successful clean installation.

`pip check` validates installed dependency compatibility; it does not perform a security audit.

### Recommended stronger locking process

1. Declare direct requirements.
2. Resolve them in the supported environment.
3. Generate exact transitive pins with artifact hashes using reviewed tooling.
4. Test clean installation across the supported matrix.
5. Record dependency and tooling versions.
6. Install with hash enforcement.
7. Review and rerun tests for every lock update.

Once a hash-based lock is available, replace installation with:

```
python -m pip install \
  --require-hashes \
  -r requirements.lock
```

Do not add `--require-hashes` to the current lock file: it does not contain hashes.

## 6. Current verification status

| Item | Status |
|---|---|
| Existing 22-method schema suite | Previously passed |
| Existing dependency versions | Recorded |
| Current setup add-on | Generated |
| Enhanced multi-stage Dockerfile | Proposed |
| Enhanced matrix workflow | Proposed |
| Docker build | Not executed |
| Matrix compatibility | Not executed |
| Hash-verified dependency installation | Not established |
| Repository changes | None made |

A passing CI result would support the synthetic schema-test evidence. It would not establish live API, IAM, SIS synchronization, PostgreSQL concurrency, or institutional activation readiness.

⁂

1. https://docs.github.com/actions/guides/building-and-testing-python